COMP 581: Introduction to Robotics

Spring 2026
Instructor: Daniel Szafir (daniel.szafir@cs.unc.edu)
Teaching Assistant: Thayer Hicks (thayerh@cs.unc.edu)

Lab 3: Moving Around an Obstacle

Deadline

- Lab pre-flection due **Friday, March 27 at 11:59pm**. To pass this part of the lab, please submit your answers in a group submission on GradeScope (see the Lab 3 Pre-flection assignment link on Canvas).
- Lab simulation: **no due date**. Turning in working code for the Webots simulation is optional for this lab and thus does not have a due date.
- Robot lab due **Monday, April 6 at the beginning of class (11:15 am)**. Before the due time, each team should submit their final code by committing and pushing to GitHub; see specific instructions below**.** For the late policy, see the course syllabus.
- Lab reflection due **Wednesday, April 8 at 11:59pm**. To pass this part of the lab, please submit your answers in a group submission on GradeScope (see the Lab 3 Reflection assignment link on Canvas).

The Objective

You will create a robot that determines the location of an unknown obstacle, traces the boundary of the obstacle, and returns to where it started.

Robot Design Requirements

Your robot must be built only using pieces from your Lego Mindstorms kit. You should design your robot such that the dark gray center button may be pressed without moving the robot and such that pushing the button does not disrupt any of the sensors. Your robot should remain in one piece at all times, its diameter with respect to the ground should not exceed 40 cm, and its diameter shall not change substantially during the task. All measurements will be made with respect to the ***measuring point*****:** a distinct point of your choosing that is on the **frontmost surface** of your robot. More specifically, you will place your robot on the ground behind a starting line drawn on the ground, such that your chosen measuring point is **directly above a point on the starting line marked by a star**, and no part of your robot is permitted to be in front of the starting line. Your robot should be designed to accomplish the task autonomously without any human intervention (except for pushing the dark gray center button as required).

Setup

We have created a Webots simulation library that will enable you to write EV3 MicroPython compatible code in Webots to control a simulated Lego robot in a world that mimics the Lab 3 environment. To download everything you need for this simulation and for deploying your program to the real robot:

1. Log into your GitHub account and then accept the GitHub Classroom assignment by clicking this link: https://classroom.github.com/a/9mPHo8W-
2. The link will take you to a page where you can accept the assignment on GitHub.
3. After accepting the assignment, it should create the repository for you and your team partner automatically based on the teams created in Lab 1. If you have any difficulty with this, please let us know.
4. The repository will be created and set up in a similar manner as in Labs 1&2. Please view the repository README for some additional details specific to this lab. You should now be able to edit the `main.py` controller file within the controllers folder in Visual Studio Code and see the updates to the robot's behavior when you run the Webots simulation.
    - Always remember to reset the Webots time back to zero.
5. Your job is now to write the main.py controller to solve the challenge described in the next section below and deploy your solution on your physical robot.

The Task

In this lab, your robot must demonstrate the capacity for autonomous navigation around an obstacle. The task in this lab is to be completed in a single run, without picking up your robot.

Pre-flection
Before programming anything, first read through the task below and make sure you understand it. Then, work with your lab partner (or complete separately and then compare answers) to fill in the Lab 3 Pre-flection worksheet linked in the Lab 3 Pre-flection assignment on Canvas and submit this through GradeScope (only one submission needed per group).

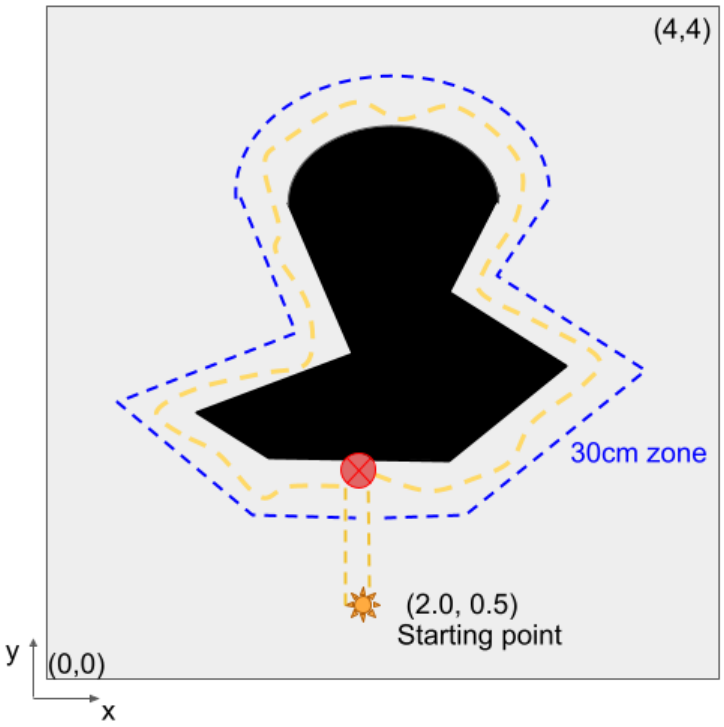
Object Navigation Task:

You will place your robot behind a starting line such that its measuring point will be on a specific *starting point* on the starting line. Directly in front of your robot will be an obstacle. The front wall of the obstacle (i.e., the wall of the obstacle facing your robot when it is at the starting point) will be located between 30 cm and 1 m away from the starting point. The point directly in front of your robot and 20 cm away from the obstacle's front wall will be known as the *hit point*.

To begin, you will push the dark gray center button on your robot. Your robot should then move straight forward until it locates the obstacle. To indicate that the robot has found the obstacle, it should beep when it is less than 30 cm of (or in contact with) the obstacle. The robot is now near the hit point. There is no need for your robot to stop, although it may. Your robot should then turn right and travel completely around the obstacle with its measuring point staying within 30 cm from the obstacle at all times. After the robot returns to the hit point, it should beep and it should turn away from the obstacle and return to the start point.

In this lab you can assume that the obstacle is a closed shape consisting of straight and curved segments. It is not necessarily a convex shape. It will be made of bumpable, ultrasonic reflective walls that are at least 17 cm tall. The perimeter of the obstacle will be between 1 m and 6 m.

All of the robot motion must be completed within a 4x4 meter square workspace without leaving the workspace. The starting point is at coordinate (2.0 m, 0.5m) The task must be completed in under 2 minutes. The figure below illustrates an example of a possible environment that complies with the rules:



The environment in your actual run may differ from this figure. In the environment above, the robot starts at the starting point (orange). The obstacle is shown in black. The wall shapes and the angles of any bend are unknown in advance. The red hit point marks the spot at which the robot should reach the obstacle and, after surrounding the obstacle, from which it should return to the start point. The dotted blue line represents a 30 cm distance to the obstacle. The yellow dashed line shows a possible robot trajectory that would receive full credit if completed in the allotted time and assuming the robots stops at the start point.

Reflection

After finishing the lab, work with your lab partner to fill in the Lab 3 Reflection worksheet linked in the Lab 3 Reflection assignment on Canvas and submit this through GradeScope (only one submission needed per group).

Grading Rubric

- **(5 Points) Pre-flection:**
    - Effort: 5 points. Points will be awarded for submitting the Pre-flection on time and for making a reasonable effort to answer each question.
- **(0 Points) Simulator:**
    - Turning in working code for the Webots simulator is *optional* for this lab. We still recommend doing it as it will likely save you time overall. However, the choice is yours.
- **(90 Points) Real Robot Attempt:**
    - Effort: 10 points. You will receive these points by:
        - Showing up to the lab on time: 5 points
        - Having your robot built and ready to attempt the course: 3 point
        - Pointing out your robot's measuring point according to the definition given in this document: 2 point
    - Robot Task: 80 points, with a breakdown as follows:
        - Find the hit point: 15 points. Points will be awarded for finding the obstacle and beginning to follow its wall. You will receive these points if your robot beeps when it is within 30 cm of (or in contact with) the obstacle.
        - Percentage traced: 25 points. Points will be awarded based on the percentage of the obstacle's perimeter that the robot successfully traced while staying inside the required distance from the obstacle wall.
        - Return to hit point: 15 points. Points will be awarded if the robot announces with a 'beep' that it has surrounded the obstacle and if it turns away from the wall.
        - Return to start point: 25 points. Points will be awarded for returning to and stopping at the start point. Points will be awarded based on accuracy (i.e., how close the robot's measuring point is to the start point when the robot stops). Full points will be awarded if the robot stops inside the goal zone, which is a circle of radius 5cm around the start position. Partial credit will be given based on distance to this goal zone.
- **(5 Points) Reflection:**
    - Effort: 5 points. Points will be awarded for submitting the Reflection on time and for making a reasonable effort to answer each question.

Penalties:

- There will be a -10 point penalty if, after crossing the start line, your robot exits the feasible workspace.
- If your robot surrounds the obstacle more than once, there will be a -5 point penalty per additional round.
- There will be a -10 point penalty if more than 2 minutes elapsed from when the dark gray center button is pushed at the start location until the robot stops. If the robot is still

moving after 2:30 minutes, then the instructor can stop the run and no additional points can be earned.

Code

To complete this lab, you will use a subset of the EV3 MicroPython API. You may only use the following modules:
ev3brick
ev3devices
parameters
tools
media
You may *not* import, use, or copy the "robotics" module, or any of its included classes or functions, in your program.

A robot relying on a module not allowed as stated above will be disqualified. Also, any robot using Wi-Fi or Bluetooth communications in any way will be disqualified.

You should submit a zip file of your Python source code files on GradeScope. The zip file should include all the Python source code files you wrote (which should have a .py file extension). In the comments at the top of the Python source file containing your main .py file, please include the names and PIDs of all team members.

You are free to discuss concepts in the course with other students or AI tools (e.g., ChatGPT) in natural language, but you should not share any programming code with others and you must write all the robot's programming code yourself. If you access any sources other than the textbook or course material posted on Canvas, you must cite them in comments at the top of your code. You must fully understand your code and be able to reproduce the algorithmic approach without references if asked. The Honor Code is in effect for these policies.

Good Luck and Have Fun!